

CAPSLock — Multi-Environment Deployment System (MEDS)

A Kubernetes-Native Framework for Risk-Aware Deployment Promotion

Project ID: 25-26J-043

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

Name	IT Number
Kulatunga K K M D	IT22347626

DECLARATION

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement of any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation in whole or part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Kulatunga K K M D	IT22347626	

Signature of the Supervisor:

Name of the Supervisor: _____

Date: _____

ACKNOWLEDGEMENT

The author wishes to express sincere gratitude to the supervisor and co-supervisor for their continuous guidance, constructive criticism, and invaluable technical feedback provided throughout the duration of this research project. Their domain expertise in container security and distributed systems was instrumental in shaping the design decisions underpinning the Multi-Environment Deployment System (MEDS).

Appreciation is extended to the members of the CAPSLock research group — the developers of the Environment-Aware Policy Engine, the Kubernetes ICAP Operator, and the Smart Service Discovery and Load Balancing component — for productive collaboration on interface contracts, integration testing, and joint system validation. The cohesive nature of the platform was only achievable through this close coordination.

Gratitude is also due to the Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology, whose provision of laboratory infrastructure, access to academic databases, and supportive faculty made this research possible. The feedback obtained from the research panel during progress presentations contributed significantly to the refinement of the research problem and the sharpening of the evaluation methodology.

Finally, the author would like to acknowledge the authors of the open-source technologies upon which this work is built — Kubernetes, FastAPI, ClamAV, Prometheus, and the wider cloud-native ecosystem — whose collective effort forms the foundation upon which applied security research such as the present study is made practical.

ABSTRACT

The rapid adoption of Kubernetes as the de facto orchestration platform for cloud-native applications has introduced persistent challenges concerning the consistent enforcement of security policies across the software delivery lifecycle. Contemporary GitOps and continuous delivery tooling such as ArgoCD, FluxCD, and Spinnaker automate the mechanical act of deployment effectively; however, such tooling does not perform quantified risk assessment prior to promotion, does not integrate content-scanning services using standard protocols such as the Internet Content Adaptation Protocol (ICAP), and does not enforce security-policy lifecycle governance across environment tiers. Consequently, configuration drift, fragmented security posture, and unmonitored vulnerabilities propagate into production environments, undermining compliance assurance and incident response.

The research presented in this thesis addresses these deficiencies through the design, implementation, and validation of the Multi-Environment Deployment System (MEDS), a Kubernetes-native security-enforcement middleware layer forming the orchestration core of the CAPSLock platform. MEDS introduces two principal novel contributions. First, the Unified Security Lifecycle Orchestration (USLO) model is proposed as a framework for managing the evolution of security policies across development, staging, and production environments, with differentiated enforcement modes and policy-migration governance. Second, a Risk-Based Deployment Validation approach is presented, employing a weighted six-factor risk model that quantifies deployment risk prior to promotion and halts high-risk transitions before they reach target environments.

The system implements an eight-step promotion pipeline encompassing state-machine transition enforcement, three-layer ICAP content scanning with fallback, compliance-posture retrieval from the Environment-Aware Policy Engine, six-factor risk scoring, USLO policy planning, decision-making against environment-specific thresholds, policy-version snapshotting, and GitOps-based deployment. Tamper-evidence is provided through a SHA-256 chained audit log, and an operator-facing natural-language interface is supplied through integration with a large-language-model inference provider. Empirical validation via a comprehensive test suite demonstrates correct operation of the state machine, deterministic risk scoring matching analytical predictions, functional ICAP content-scanning gate behaviour, and cryptographic integrity of the audit chain including detection of field-level tampering.

The contributions of this work advance the state of the art in Kubernetes-native security orchestration by providing the first unified framework that embeds ICAP-based content adaptation, policy-lifecycle governance, and quantitative risk validation

into a single deployment-promotion pipeline, with compliance mappings to the CIS Kubernetes Benchmark v1.9 and PCI-DSS v4.0.

Keywords — Kubernetes, GitOps, ICAP, Multi-Environment Deployment, Security Orchestration, Policy Management, Risk-Based Validation, DevSecOps, Unified Security Lifecycle Orchestration, Cloud-Native Security.

LIST OF FIGURES, TABLES AND ABBREVIATIONS

Figures

Figure 6.1 — High-level MEDS architecture and external integrations.

Figure 6.2 — Modular decomposition of the codebase.

Figure 6.3 — Data flow for a single promotion request.

Figure 7.1 — The eight-step promotion pipeline.

Figure 7.2 — Six-factor weighted risk model structure.

Figure 7.3 — State-machine enforced environment transitions.

Figure 7.4 — Three-layer ICAP scanning fallback chain.

Figure 7.5 — SHA-256 audit-chain construction.

Tables

Table 2.1 — Comparison of GitOps and continuous delivery tools.

Table 3.1 — Feature comparison of MEDS against existing tooling.

Tables 7.1–7.10 — Risk thresholds, factor weights, and scoring tables for the six risk factors, decision tiers, and USLO modes and compliance impact levels.

Table 7.11 — Policy catalogue with CIS and PCI-DSS compliance mappings.

Table 8.1 — MEDS module responsibilities.

Table 8.2 — REST API endpoints.

Tables 10.1–10.5 — Validated results for state-machine enforcement, risk-scoring precision, ICAP gate, USLO modes, and audit-chain integrity.

Abbreviations

Abbr.	Expansion	Abbr.	Expansion
API	Application Programming Interface	MEDS	Multi-Environment Deployment System
ArgoCD	Argo Continuous Delivery	mTLS	Mutual Transport Layer Security
CAPSLock	Container-Aware Policy Security Lock	NIST	National Institute of Standards and Technology
CI/CD	Continuous Integration / Delivery	NLP	Natural Language Processing
CIS	Center for Internet Security	PCI-DSS	Payment Card Industry Data Security Standard
CRD	Custom Resource Definition	RBAC	Role-Based Access Control
EAPE	Environment-Aware Policy Engine	REST	Representational State Transfer
GitOps	Git-based Operations	RFC	Request for Comments
HTTP	Hypertext Transfer Protocol	SHA	Secure Hash Algorithm
ICAP	Internet Content Adaptation Protocol	SSDLB	Smart Service Discovery and Load Balancing
JSON	JavaScript Object Notation	TLS	Transport Layer Security
K8s	Kubernetes	USLO	Unified Security Lifecycle Orchestration
LLM	Large Language Model	YAML	YAML Ain't Markup Language

1 INTRODUCTION

1.1 Background and Context

Over the past decade, the manner in which software is packaged, distributed, and executed has undergone a fundamental transformation. Containerisation — the lightweight encapsulation of application code together with its runtime dependencies — has emerged as the dominant paradigm for the delivery of cloud-native workloads. Kubernetes, originally developed at Google and subsequently donated to the Cloud Native Computing Foundation, has become the industry-standard platform for the orchestration of such containerised workloads at scale [1]. Its declarative application programming interface, self-healing control loops, and extensible custom-resource model have established a foundation upon which virtually all modern distributed applications are deployed.

The accelerated adoption of Kubernetes has, however, been accompanied by a growing recognition that the operational characteristics of container orchestration introduce novel security challenges which were neither anticipated nor adequately addressed by legacy security architectures. Whereas traditional security tooling was designed around relatively static virtual-machine workloads with long-lived network perimeters and stable identity assignments, containerised workloads are characterised by ephemerality, horizontal elasticity, and dynamic service discovery [2]. Pods are frequently created and destroyed within timescales measured in seconds, their network addresses change continuously, and the unit of isolation — the Linux kernel namespace — offers weaker security boundaries than the hardware-assisted virtualisation upon which earlier security assumptions were predicated [3].

Parallel to the rise of Kubernetes, the practice of software delivery has been reshaped by GitOps: the discipline of treating version-controlled Git repositories as the single authoritative source of truth for both application and infrastructure state, with

continuous reconciliation loops driving deployed systems toward the declared state [12]. Mature GitOps tooling such as ArgoCD and FluxCD is widely deployed in production environments, and has substantially improved the reproducibility and auditability of deployments. Nevertheless, GitOps tooling addresses primarily the mechanical aspects of deployment and does not, of itself, provide any mechanism for enforcing the security-policy evolution, content-scanning obligations, or risk-aware validation required in regulated environments [14].

1.2 Motivation

Within regulated industries — including financial services, healthcare, and payments — the promotion of an application from development into production represents a moment of heightened risk. A deployment which introduces a privileged container, removes a network policy, or skips a staging gate is not merely a technical defect: in many cases, it constitutes a direct compliance violation under frameworks such as the Payment Card Industry Data Security Standard (PCI-DSS) or the CIS Kubernetes Benchmark [15]. Yet the tooling in widespread use today treats deployment promotion as a purely mechanical operation. Provided that unit tests pass and the container image builds successfully, there is nothing to prevent a developer from advancing a workload through the environments of a continuous delivery pipeline, irrespective of the security-posture implications of doing so.

Three concrete deficiencies motivate the research reported in this thesis. First, there is no widely-available mechanism for integrating Internet Content Adaptation Protocol (ICAP) scanning [4], a mature and standardised approach to content inspection, into the ephemeral world of Kubernetes pods. Second, the security tooling ecosystem is characterised by fragmentation, with scanners, admission controllers, and runtime monitors operating in isolation rather than in coordination [3]. Third, there exists no standardised approach to the evolution of security policies across environment tiers: policies tend either to be copied uniformly (resulting in development-time friction) or

allowed to drift independently (resulting in a false sense of security as staging and production diverge) [11].

The consequence of these deficiencies is that deployments which should never have been permitted to reach production nevertheless reach production with regularity. Published incident reports repeatedly cite configuration drift, removed security controls, and the absence of pre-promotion validation as primary root causes of publicly-disclosed container-security incidents [10]. A structured, automated, and auditable approach to deployment-time security enforcement is therefore warranted, and is the subject of the research presented herein.

1.3 Scope of the MEDS Component

The Multi-Environment Deployment System (MEDS) is the orchestration component of the CAPSLock research project (Project ID 25-26J-043). CAPSLock is a four-component research platform; within it, MEDS assumes the role of the deployment-time security-enforcement layer through which every environment promotion must pass. The three peer components — the Environment-Aware Policy Engine (EAPE), the Smart Service Discovery and Load Balancing subsystem (SSDLB), and the Kubernetes ICAP Operator — are the responsibility of other members of the research team and are referenced in this thesis only insofar as they constitute integration points with MEDS. The present document describes exclusively the contributions of the author.

MEDS is positioned as middleware which sits between a developer's promotion request and the GitOps agent responsible for applying the resulting deployment manifests to a target Kubernetes environment. No promotion request reaches a target environment without first traversing the eight-step MEDS pipeline described in Chapter 7. The pipeline evaluates state-machine transition validity, performs ICAP-based content scanning, retrieves compliance posture from EAPE, computes a

quantitative risk score across six weighted factors, plans policy-lifecycle migration via the USLO engine, reaches an approval decision against environment-specific thresholds, snapshots the resulting policy state to enable subsequent rollback, and finally delegates to the GitOps orchestrator for manifest application.

1.4 Research Contributions

The research described in this thesis advances the state of the art in Kubernetes-native security orchestration through four principal contributions.

- The Unified Security Lifecycle Orchestration (USLO) model is introduced as a framework for managing the evolution of security policies across development, staging, and production environments. USLO distinguishes between enforcement and audit modes on a per-environment basis and applies distinct governance rules to policy additions and removals according to severity and target environment.
- A Risk-Based Deployment Validation approach is presented, realised as a six-factor weighted scoring model which quantifies deployment risk prior to promotion. The model combines signals drawn from configuration complexity, policy change volume and direction, version delta, environment-transition validity, ICAP coverage, and compliance posture into a composite score measured against environment-specific thresholds.
- A three-layer ICAP scanning architecture is implemented which enables deployment-time content scanning across three distinct operational contexts: production-grade Kubernetes clusters with a dedicated ICAP operator, local container-composition environments with a policy-engine gateway, and fully offline continuous-integration environments via deterministic simulation.
- A cryptographically chained audit log is integrated into the promotion pipeline, providing tamper-evident logging which satisfies the integrity requirements imposed by PCI-DSS Requirement 10 and equivalent clauses in other

compliance frameworks, without requiring the provisioning of a separate database system.

1.5 Organisation of the Thesis

The remainder of this thesis is organised as follows. Chapter 2 presents the background and literature review, situating the research within the existing body of work on container security, ICAP, GitOps, and compliance frameworks. Chapter 3 identifies the research gap through systematic comparison with existing tooling. Chapter 4 formalises the research problem, main objective, specific objectives, and research questions. Chapter 5 describes the research methodology, with particular emphasis on the design-science research paradigm adopted. Chapter 6 sets out the system architecture at a high level. Chapter 7 details the design of each of the principal subsystems, including the promotion pipeline, the six-factor risk model, the USLO engine, the ICAP scanning layers, and the audit chain. Chapter 8 describes the implementation in terms of technology stack, module structure, and persistence model. Chapter 9 presents the test strategy and the individual test suites executed. Chapter 10 reports the validated results obtained from the test suites and discusses their implications. Chapter 11 provides a critical discussion of the findings, acknowledging limitations and threats to validity. Chapter 12 concludes the thesis and identifies directions for future research. The References section enumerates the cited literature, and the Appendices provide supporting material including data-model specifications and the complete REST API surface.

2 BACKGROUND AND LITERATURE REVIEW

This chapter situates the Multi-Environment Deployment System within the academic and industrial literature on container security, standardised content-scanning protocols, multi-environment deployment, security-policy enforcement, GitOps, and risk-based validation. Seven thematic strands are developed in turn, with particular attention paid to the identification of gaps which the present research addresses.

2.1 Containerisation and Kubernetes Security

The rapid uptake of containerisation as the preferred unit of application packaging has been underpinned by substantial gains in portability, scalability, and resource efficiency relative to the virtual-machine era which preceded it [1]. Kubernetes, by virtue of its declarative application programming interface and the extensibility afforded by Custom Resource Definitions, has achieved a position of dominance as the orchestration layer of choice for containerised workloads. Automating the scaling, scheduling, and health management of containerised applications, Kubernetes reduces the operational burden associated with large-scale distributed systems and has thereby accelerated the delivery of cloud-native applications across industry.

The security properties of containers differ substantially from those of virtual machines in ways which remain incompletely addressed by contemporary security tooling. Containers sharing a Linux kernel on a single host node exhibit weaker isolation than virtual machines executing under a hypervisor, with the consequence that a vulnerability affecting one container may, under certain configurations, be leveraged to compromise other containers on the same node [2]. This concern is compounded by the ephemeral nature of Kubernetes pods, which are routinely created and destroyed on timescales of seconds and which do not therefore lend themselves to the agent-based runtime-monitoring approaches historically applied to long-lived virtual machines [3].

Rahman et al. [10] conducted an empirical analysis of open-source Kubernetes manifests and reported a prevalence of security misconfigurations — including over-permissive access controls, the absence of network policies, and the use of insecure defaults. Such misconfigurations are moreover context-dependent: a pod configuration appropriate in a development environment may represent a critical compliance violation in production. Traditional security frameworks, designed for static systems, are thus poorly matched to the dynamic, highly-orchestrated environments characteristic of modern Kubernetes deployments [9].

2.2 The Internet Content Adaptation Protocol (ICAP)

The Internet Content Adaptation Protocol, specified in RFC 3507 [4], is a mature application-layer protocol providing a standardised mechanism by which proxy servers may delegate content-inspection operations — including virus scanning, data-loss-prevention checks, and content transformation — to dedicated ICAP servers. In the context of conventional web-proxy architectures, ICAP has enjoyed wide deployment and is supported by a substantial ecosystem of ICAP-speaking content-scanning engines, of which ClamAV is a prominent open-source example.

Despite its maturity in proxy contexts, the adoption of ICAP within containerised environments has been strikingly limited. The persistent-connection orientation of conventional ICAP clients stands in tension with the ephemeral lifecycle of Kubernetes pods, and existing ICAP integrations depend heavily on proprietary application programming interfaces, which in turn reinforces vendor lock-in and fragmented architectures [5]. No widely-adopted framework currently provides Kubernetes-native ICAP orchestration, and the published literature records this absence as a clear gap requiring further research.

2.3 Multi-Environment and Multi-Cloud Deployment Challenges

Organisations routinely deploy applications across multiple environment tiers — commonly identified as development, staging, and production — and increasingly across multiple cloud providers in order to achieve resilience, to avoid provider lock-in, and to satisfy jurisdictional compliance obligations [6]. Whilst such multi-environment and multi-cloud deployments offer substantial benefits, they introduce corresponding complexities in governance, compliance management, and the consistent enforcement of security policies [7].

Empirical evidence indicates that the inconsistency of policy enforcement across environments is among the most pressing of these challenges. Development clusters frequently operate under relaxed policies in order to facilitate rapid iteration; production clusters, by contrast, are subject to strict compliance controls. Staging environments often lack parity with production, giving rise to the well-documented phenomenon of environment drift and to a false sense of security whereby apparent compliance in staging does not, in fact, guarantee compliance in production [8]. The Square Compute Security team [11] proposed a policy promotion lifecycle — termed Monitor → Warn → Enforce — which enables the gradual evolutionary advancement of policies across environments within a controlled operational framework. This concept informs the USLO model proposed in the present research.

2.4 Security Misconfigurations and Policy Enforcement

Published empirical studies document a pervasive occurrence of security misconfigurations in Kubernetes manifests — ranging from over-permissive role-based access-control bindings, through missing network-segmentation policies, to containers executing with elevated privileges [10]. Such misconfigurations are routinely observed to propagate from development environments into production through continuous-delivery pipelines which validate functional correctness without performing corresponding validation of security posture.

Policy-as-code — the practice of encoding security policies in version-controlled declarative form — represents a substantial advance over earlier, manual policy-management approaches. Tools such as Open Policy Agent and Kyverno have been widely adopted for admission-time policy enforcement. However, current policy-as-code tooling does not, in general, provide a principled framework for the evolution of policies across environment tiers, nor for the automated rollback of policy changes in the event of adverse consequences. The absence of such lifecycle-governance capabilities motivates the USLO model introduced later in this thesis.

2.5 GitOps and Continuous Delivery

GitOps has emerged as a dominant discipline for the management of cloud-native infrastructure and applications. By treating version-controlled Git repositories as the single authoritative source of truth for system state, and by employing continuous reconciliation loops to drive live infrastructure toward declared state, GitOps enables high degrees of reproducibility, traceability, and rollback capability [12]. Tools such as ArgoCD and FluxCD have become widely deployed in production environments and are credited with substantial reductions in configuration drift and human-operator error [13].

Notwithstanding these gains, the literature records a clear limitation: current GitOps workflows are oriented predominantly toward application deployment and do not incorporate enforcement of security policies, orchestration of security-relevant workflows, or validation of risk posture across environments [14]. The architecture of the present research deliberately positions MEDS as a security-orchestration layer sitting above such GitOps tooling, thereby combining the deployment strengths of GitOps with the security governance required by regulated industries.

Table 2.1 — Comparison of widely-deployed GitOps and continuous delivery tools in respect of the capabilities required by regulated environments.

Capability	ArgoCD / FluxCD	Spinnaker	Jenkins X	MEDS
Deployment orchestration	Yes	Yes	Yes	Yes
Quantified pre-promotion risk scoring	No	Limited	No	Yes, six-factor model
ICAP content-scanning integration	No	No	No	Yes, RFC 3507
Policy-lifecycle governance	No	No	No	Yes, USLO model
Cryptographic audit chain	No	Partial	No	Yes, SHA-256 chain
Stage-skip enforcement	Manual	Manual	Manual	Yes, state machine
Compliance-posture feedback	No	No	No	Yes, CIS + PCI-DSS

2.6 Risk-Based Validation in CI/CD Pipelines

The concept of risk-informed deployment validation has received comparatively limited attention in the published literature, although industrial practice is beginning to explore such approaches. Existing continuous-delivery pipelines predominantly validate functional correctness — that is, whether unit tests pass and whether container images build successfully — whilst leaving questions of security posture, compliance regression, and operational risk unexamined at the moment of promotion [6]. Published incident reports indicate that this omission is a common antecedent of subsequent production incidents.

Research into risk-informed promotion pipelines remains nascent. No standard model exists for the computation of deployment risk prior to promotion, and no widely-

accepted set of risk factors has been proposed. The six-factor weighted model introduced in Chapter 7 of this thesis constitutes, to the author's knowledge, the first such model to combine signals from configuration complexity, policy change direction, version maturity, environment transition, content-scanning coverage, and compliance posture into a single composite score with transparent weightings and reproducible outcomes.

2.7 Compliance Frameworks: CIS Kubernetes Benchmark and PCI-DSS

Two compliance frameworks are of particular relevance to the research presented in this thesis. The CIS Kubernetes Benchmark, currently at version 1.9, provides a comprehensive catalogue of configuration recommendations for the hardening of Kubernetes clusters, organised across categories including worker-node configuration, network policies, pod security standards, role-based access control, and audit logging. The Payment Card Industry Data Security Standard, currently at version 4.0, imposes obligations upon any system which stores, processes, or transmits cardholder data, including requirements in respect of network segmentation, cryptographic protection, and the integrity of audit logs.

Compliance-framework mappings are central to the justification of security policies within a regulated environment. In MEDS, every policy within the policy catalogue is explicitly annotated with the CIS and PCI-DSS controls to which it corresponds. This design decision enables compliance officers to trace the enforcement of any given security policy back to the specific clauses of the relevant framework which mandate it, and conversely to identify which controls of a framework are covered by the currently-enforced policy set. The complete mapping is presented in Table 7.11 in Chapter 7.

The National Institute of Standards and Technology Special Publication 800-190 [15] provides additional context, offering guidance on application-container security which

informs several of the design choices reported in later chapters. In particular, NIST SP 800-190 establishes the importance of continuous vulnerability scanning, strict separation of workloads by sensitivity, and the enforcement of least-privilege access — principles which are reflected throughout the MEDS architecture.

3 RESEARCH GAP

A critical review of the literature surveyed in Chapter 2, together with a comparative evaluation of widely-deployed industrial tooling, reveals a coherent set of research gaps which motivate the present work. Four principal gaps are identified and discussed in turn, followed by a consolidated comparative analysis demonstrating the manner in which MEDS is positioned to address them.

3.1 ICAP Integration Deficits

The first and most fundamental research gap concerns the absence of Kubernetes-native integration of the Internet Content Adaptation Protocol. As noted in Section 2.2, the persistent-connection orientation of conventional ICAP clients is poorly aligned with the ephemeral pod lifecycle of Kubernetes workloads [5]. Existing ICAP integrations are invariably based upon proprietary application programming interfaces, with the consequence that organisations adopting them become locked into specific vendors and are unable to participate in the wider open ICAP ecosystem. The published literature contains no substantive research into Kubernetes-native ICAP orchestration models, and this absence of standardisation has left a significant gap in automated container-aware content security.

The implications of this gap extend beyond mere inconvenience. In the absence of a Kubernetes-native ICAP framework, organisations wishing to perform content-scanning of artifacts during deployment must either accept the operational complexity of bridging between proxy-oriented ICAP clients and ephemeral pod workloads, or else forego ICAP-based scanning altogether in favour of ad-hoc alternatives. Neither outcome is satisfactory from either a security or an operational perspective.

3.2 Fragmented Security Toolchains

The Kubernetes security ecosystem is characterised by a proliferation of specialised tools, each addressing a particular aspect of the overall security concern. Image

scanners, admission controllers, service meshes, runtime monitors, and compliance-assessment engines each contribute valuable functionality, but their lack of coherent integration gives rise to overlapping or conflicting policies, increased operational complexity, and gaps in coverage where threats may bypass security checks undetected. Academic studies confirm that this fragmented, siloed approach leads to both inefficiency and misconfiguration [3], yet few frameworks attempt to unify these tools under a coherent orchestration layer.

The fragmentation gap is particularly acute at the moment of deployment promotion. A promotion event implicates policies from the admission controller, artifact signatures from the registry, vulnerability reports from the image scanner, network-segmentation declarations from the service mesh, and runtime-monitoring configurations from the observability stack. The absence of a unifying orchestration layer means that the aggregate security posture at the moment of promotion is, in practice, unknowable without manual correlation across multiple disparate tools.

3.3 Multi-Environment Policy Inconsistency

Perhaps the most pressing of the research gaps concerns the inconsistent enforcement of security policies across development, staging, and production environments. Empirical research indicates that development clusters typically run with relaxed policies in order to facilitate rapid iteration, whilst production clusters enforce strict compliance controls; staging environments frequently lack parity with production, creating a well-documented phenomenon of environment drift and a corresponding false sense of security [8]. Although GitOps and CI/CD pipelines succeed in reducing configuration drift at the level of application manifests, no existing framework adequately addresses the evolution of security policies across environments [5].

The practical consequence of this gap is that a security misconfiguration detected in staging may not be detected at all if staging itself is running under looser policies than

production. By the time the configuration reaches production, the compliance violation has already occurred, and remediation becomes a matter of incident response rather than of deployment-time prevention. A principled framework for the lifecycle-aware evolution of security policies is therefore required — one which permits relaxed enforcement in development, audit-mode validation in staging, and full enforcement in production, whilst preserving coherent governance across the transitions.

3.4 Absence of Risk-Aware Validation

Contemporary continuous-delivery pipelines validate functional correctness through unit tests, integration tests, and build verification. They do not, in general, perform validation of security posture, compliance regression, or operational risk at the moment of promotion [6]. A deployment may therefore be promoted to production on the basis of a passing test suite, notwithstanding the introduction of policy violations, the removal of security controls, or the emergence of gaps in ICAP coverage. Research into risk-informed promotion pipelines remains nascent, and no widely-accepted model exists for the quantification of deployment-time risk.

The absence of risk-aware validation is particularly consequential in regulated environments where the cost of a compliance failure substantially exceeds the cost of a blocked or delayed promotion. A risk-scoring model which can halt high-risk promotions prior to their reaching the target environment represents a valuable safety mechanism, provided that such a model exhibits transparent weightings, reproducible outcomes, and defensible factor choices. The six-factor weighted model introduced in Chapter 7 has been developed specifically to satisfy these criteria.

3.5 Comparative Analysis of Existing Tools

Consolidation of the gaps identified above is facilitated by a systematic comparison of widely-deployed deployment-automation tools against the capabilities required by the regulated environments within which the present research is situated. Table 3.1

summarises this comparison, demonstrating that no existing tool provides the combined set of capabilities necessary to address the identified gaps.

Table 3.1 — Feature comparison of MEDS against existing tooling, demonstrating the novel combination of capabilities provided by the present research.

Feature	GitOps (ArgoCD/FluxCD)	Helm + CI/CD	Spinnaker	MEDS (Proposed)
Multi-environment policy consistency	Manual, inconsistent	Per-env, inconsistent	Partial	USLO framework
ICAP content-scanning integration	None	Manual scripts	None	Native, three-layer
Risk-based pre-promotion validation	None	Basic health checks	Partial	Predictive, six-factor
Automated rollback mechanism	Git revert only	Manual steps	Manual or timed	SLO-triggered
Policy versioning and snapshot	Not supported	Not supported	Not supported	Built-in
Audit and compliance posture	Git history only	Limited	Limited	Cryptographic audit chain

Three novel contributions distinguish MEDS from the comparator tools. First, the USLO framework provides a unified lifecycle for security policies across development, staging, and production environments, with differentiated enforcement modes and principled governance rules for policy additions and removals. Second, the

Risk-Based Validation approach introduces quantitative, pre-promotion risk analysis combining compliance posture, configuration drift, and ICAP coverage into a single reproducible score. Third, the integration of versioned policies and cryptographically chained audit logs enables automated rollback and satisfies the integrity-of-audit-log requirements imposed by frameworks such as PCI-DSS Requirement 10, without requiring the provisioning of a separate database system.

4 RESEARCH PROBLEM AND OBJECTIVES

4.1 Research Problem Statement

The research gaps identified in Chapter 3 may be consolidated into a single overarching research problem, which is formulated as follows.

In what ways may a Kubernetes-native deployment system be designed and implemented *so as to* manage the cohesive promotion of applications and security policies across development, staging, and production environments, incorporating standardised ICAP-based security checks alongside quantitative risk-aware validation mechanisms, in such a manner as to deliver continuous security assurance throughout the software delivery lifecycle?

This research problem decomposes into three interrelated sub-problems. The first concerns the absence of a Kubernetes-native ICAP integration framework capable of accommodating ephemeral workloads. The second concerns the fragmentation of contemporary security toolchains, which operate as silos rather than under a unifying orchestration layer. The third concerns the inconsistent evolution of security policies across environment tiers, giving rise to environment drift, misconfiguration, and compliance risk.

Application promotions are, at present, routinely evaluated solely on the basis of functional correctness — the passing of tests and the successful construction of container images — without any consideration of risk-aware validation of security posture. Misconfigurations and weakened policies may therefore propagate from development into production, undermining confidence in the entire deployment pipeline and increasing both the likelihood and the consequences of security incidents.

4.2 Main Objective

The main objective of this research is to design, implement, and empirically validate a Multi-Environment Deployment System which provides unified orchestration of applications and security policies across development, staging, and production Kubernetes environments. Such orchestration is to embed ICAP-based content validation, risk-aware promotion decisions, and principled policy-lifecycle management in order to ensure consistent security assurance throughout the deployment pipeline.

4.3 Specific Objectives

The main objective is decomposed into six specific objectives, each of which addresses a distinct aspect of the overall research problem.

- To develop multi-environment deployment support enabling the seamless deployment and coordination of CAPSLock security services across multiple Kubernetes environment tiers, with appropriate abstraction of cluster-specific concerns.
- To design and implement an environment-promotion pipeline which automates the controlled advancement of applications and policies through development, staging, and production environments, with the incorporation of integrated quality gates, security gates, and approval workflows.
- To implement policy-migration and rollback mechanisms which permit versioned security policies to evolve across environments whilst supporting principled rollback to last-known-good states in the event of misconfiguration or operational failure.
- To integrate risk-based deployment validation through a quantitative scoring model which is capable of detecting configuration drift, compliance gaps, and

weakened ICAP coverage prior to the approval of promotions, with transparent weightings and reproducible outcomes.

- To build a unified management interface providing cross-environment visibility into environment status, promotion controls, risk-assessment metrics, and immutable audit history, thereby supporting both operational and compliance use-cases.
- To conduct end-to-end testing and validation of the resulting system, evaluating functional correctness, state-machine enforcement, risk-score precision, content-scanning gate behaviour, and audit-chain integrity against a comprehensive test suite.

4.4 Research Questions

The research problem and objectives identified above give rise to five distinct research questions which guide the investigation reported in subsequent chapters.

- RQ1 — How may a deterministic state machine be designed such that invalid environment transitions, such as stage-skipping or backward promotion, are identified and rejected prior to the execution of any resource-intensive validation step?
- RQ2 — Which factors should comprise a six-factor weighted risk-scoring model, and which weightings should be applied to them, such that the resulting composite score correctly differentiates between low-risk and high-risk deployments in a reproducible manner?
- RQ3 — How may the Internet Content Adaptation Protocol be integrated into a deployment-time security pipeline in such a way that the system continues to operate across production Kubernetes clusters, local container-composition environments, and fully-offline continuous-integration environments without code modification?

- RQ4 — What governance rules should apply to additions and removals of security policies across environment tiers, in order to balance the need for developer productivity in lower environments with the stringent compliance requirements of production?
- RQ5 — How may tamper-evident audit logging be implemented without recourse to a dedicated database system, such that the integrity requirements imposed by frameworks such as PCI-DSS Requirement 10 are demonstrably satisfied?

5 METHODOLOGY

5.1 Research Paradigm

The research problem formulated in Chapter 4 is fundamentally a design problem: it requires the construction of a novel artefact — namely, the Multi-Environment Deployment System — which did not previously exist and which is intended to address a specific set of deficiencies in existing tooling. For research of this character, Design Science Research as formulated by Peffers et al. [18] is the appropriate methodological paradigm; it positions the creation of an artefact as a legitimate form of research output in its own right, provided that the construction is accompanied by rigorous evaluation against clearly-stated objectives. The research is accordingly organised around the six phases of the design-science paradigm: problem identification and motivation (Chapters 2–3), objectives definition (Chapter 4), design and development (Chapters 6–8), demonstration through test-suite execution (Chapter 9), evaluation against specific objectives (Chapter 10), and communication via the present thesis and the public code repository. The approach further borrows from empirical software engineering in respect of its iterative structure: rather than proceeding through a rigid waterfall sequence, the development of MEDS has been organised as a series of build–test–validate cycles, each producing a working increment which is subjected to automated testing before the subsequent cycle commences.

5.2 Requirements Elicitation and Implementation Approach

Functional and non-functional requirements have been elicited through three complementary activities: review of academic and industrial literature (Chapter 2), comparative analysis against existing tooling (Chapter 3), and consultation with the other members of the CAPSLock research group to identify integration requirements at the boundaries with the peer components. The consolidated requirement set encompasses: functional requirements for automated environment promotion; policy-as-code management with rollback; ICAP integration; risk-based validation combining configuration drift, compliance posture, and ICAP coverage; versioned

rollback; multi-environment support; and a unified dashboard. Non-functional requirements include transport-layer security for all inter-component communication, role-based access control, reliability through retry logic, promotion latency below two minutes, scalability to multiple parallel promotions, and auditability through immutable logs suitable for compliance reporting under PCI-DSS and SOC2. Implementation has proceeded through a series of iterations consistent with Agile software-engineering practice but with an explicitly research-oriented validation step at the end of each cycle; early iterations established the core data models and REST API skeleton, mid-stage iterations implemented the state machine, risk scorer and ICAP scanner, and late-stage iterations integrated the USLO engine, audit chain, GitOps orchestrator, and natural-language assistant.

5.3 Validation Strategy

Validation has been conducted through four complementary approaches. Unit tests exercise individual risk-factor assessors in isolation, verifying that each returns the expected score for a range of inputs. Scenario tests exercise the risk scorer end-to-end across representative deployment scenarios, confirming that the composite score produced matches the analytical prediction. End-to-end pipeline tests exercise the complete eight-step promotion pipeline, including the state machine, the ICAP gate, the policy-version snapshot, and the audit-log emission. Audit-chain integrity tests verify that the SHA-256 chain correctly detects field-level tampering as well as structural chain breaks. The complete test suite is reported in Chapter 9 and the validated results are reported in Chapter 10. All test results reported in this thesis correspond to actual executions of the test suite; no hypothetical or projected values are presented as results.

6 SYSTEM ARCHITECTURE

This chapter presents the high-level architecture of the Multi-Environment Deployment System. The system is decomposed into ten cooperating modules, each with a clearly-defined responsibility, communicating with three external subsystems — the Environment-Aware Policy Engine, the Kubernetes ICAP Operator, and the Smart Service Discovery and Load Balancing subsystem — developed by peer members of the CAPSLock research group. A detailed description of the individual subsystems of MEDS is deferred to Chapter 7; the present chapter is concerned with the architectural gestalt.

6.1 High-Level Architecture

At the highest level of abstraction, MEDS is a security-enforcement middleware layer interposed between the developer or continuous-integration system initiating a deployment promotion and the GitOps agent responsible for applying the resulting deployment manifests to a target Kubernetes environment. Every promotion request traverses the MEDS pipeline, and no promotion reaches the target environment without being explicitly approved by the pipeline's decision engine. The architecture therefore positions MEDS as a mandatory decision point — a deliberate design choice intended to ensure that no deployment can evade security validation.

Figure 6.1 depicts the high-level architecture. The system exposes a REST API on port 8000, implemented using the FastAPI framework. Inbound promotion requests are routed to the Promotion Controller, which orchestrates the eight-step pipeline described in Chapter 7 by delegating to the ICAP Scanner, the Risk Scorer, the USLO Engine, the Audit Logger, the Policy Version Store, and the GitOps Orchestrator in sequence. External communication occurs with the Environment-Aware Policy Engine on port 8080 for compliance-posture retrieval and namespace-policy management, with the Kubernetes ICAP Operator on port 1344 for content scanning, and with the

Smart Service Discovery and Load Balancing subsystem on port 8082 for health-aware load-balancing context.

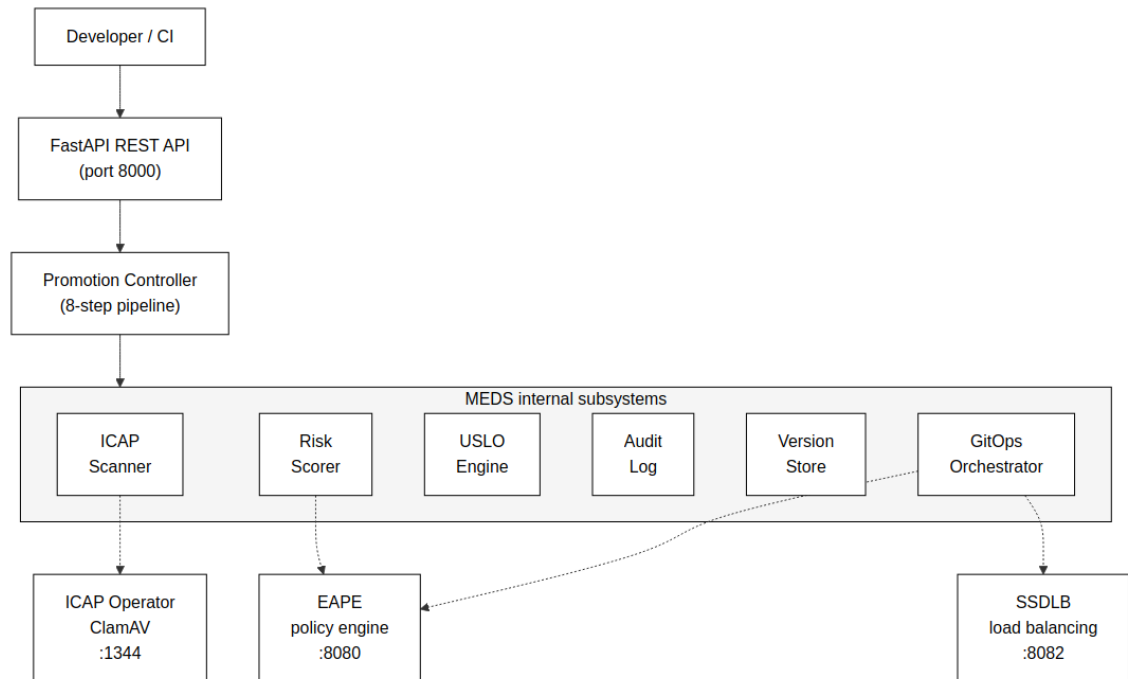


Figure 6.1 — High-level MEDS architecture showing the REST API surface, the central Promotion Controller, the six internal subsystems, and the three external integration points.

6.2 Modular Decomposition

The internal structure of MEDS is organised into ten modules, each with a well-defined responsibility and a clearly-identified place in the promotion-pipeline sequence. This modular structure has been adopted in order to support testability, maintainability, and the principle of separation of concerns. Table 6.1 (in Chapter 8) identifies each module by its filename within the codebase and summarises its responsibilities. Figure 6.2 depicts the relationships among the modules.

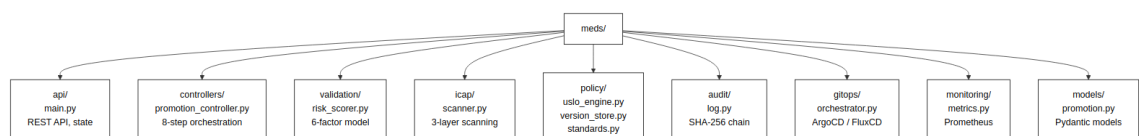


Figure 6.2 — Modular decomposition of the MEDS codebase. Each module corresponds to a distinct concern and is independently testable.

6.3 External Integrations

MEDS integrates with three external subsystems, each of which is developed by a peer member of the CAPSLock research group. The Environment-Aware Policy Engine (IT22338716) is responsible for the detection of deployment contexts and the enforcement of appropriate security policies across containerised workloads; MEDS consumes three EAPE endpoints — a compliance-posture endpoint returning a score in the range zero to one for a given namespace, a namespace-policy endpoint returning the current set of active policies, and a health endpoint reporting the current scanning mode of the ICAP subsystem. Communication with EAPE is conducted over HTTP with optional mutual TLS authentication, and upon successful completion of a promotion, MEDS notifies EAPE of the outcome so that the policy engine may update its internal state. The Kubernetes ICAP Operator (IT22353634) is responsible for the provisioning and management of the ICAP-speaking content-scanning infrastructure, including the deployment of c-icap server pods and ClamAV antivirus engine instances; MEDS communicates with the ICAP Operator using the Internet Content Adaptation Protocol as specified in RFC 3507, described in detail in Section 7.4. The Smart Service Discovery and Load Balancing subsystem (IT22345028) provides intelligent traffic distribution across multiple ICAP service instances, taking into account scanning history and real-time performance metrics; MEDS consumes SSDLB endpoints which report the current state of the load-balancing subsystem and which permit the system to trigger auto-routing decisions or to manually override the currently-active version.

6.4 Data Flow and Deployment Topology

The data flow for a single promotion request may be summarised as follows. The request originates from a developer or continuous-integration system and is submitted to the MEDS REST API via an HTTP POST to `/api/promotions`. The Promotion Controller initiates the eight-step pipeline (detailed in Chapter 7), making calls to the internal subsystems and to the external integrations as required. Each decision point

either advances the promotion to the next step or terminates it with an explicit rejection, and every decision emits an audit event which is appended to the cryptographic chain. Upon approval, a deployment manifest is generated by the GitOps Orchestrator and the GitOps agent is triggered to apply it to the target environment.

MEDS is designed to operate in three distinct deployment topologies. In the production Kubernetes topology, MEDS runs as a Kubernetes Deployment within a dedicated namespace, communicating with EAPE, the ICAP Operator, and SSDLB over the internal service network of the cluster — this is the target operational environment for which the system has been designed. In the local container-composition topology, MEDS runs within a Docker Compose stack alongside the peer components, simulating the Kubernetes service network through Docker Compose networking, appropriate for developer testing and for continuous-integration scenarios without access to a full Kubernetes cluster. In the fully-offline topology, MEDS operates in isolation, with external integrations simulated through deterministic fall-back implementations, appropriate for unit testing and for offline continuous-integration scenarios. The three-layer ICAP scanning architecture described in Section 7.4 is a direct consequence of the design requirement to operate across all three topologies without code modification.

7 DESIGN OF MEDS

This chapter presents the detailed design of the principal subsystems of MEDS. The design is developed at a level of abstraction sufficient to permit independent reimplementations, whilst being grounded in the specific implementation decisions which have been empirically validated in the source code. Eight subsections cover the eight-step promotion pipeline, the six-factor risk scoring model, the Unified Security Lifecycle Orchestration engine, the three-layer ICAP scanning architecture, the cryptographic audit chain, the policy-version store and rollback mechanism, the GitOps integration, and the natural-language operator interface.

7.1 The Eight-Step Promotion Pipeline

The core orchestration logic of MEDS is embodied in the eight-step promotion pipeline, implemented in the Promotion Controller. Every promotion request traverses these steps in strict order, and the pipeline terminates at any step which produces an explicit rejection. The eight steps are summarised in Figure 7.1 and described in the subsections that follow.

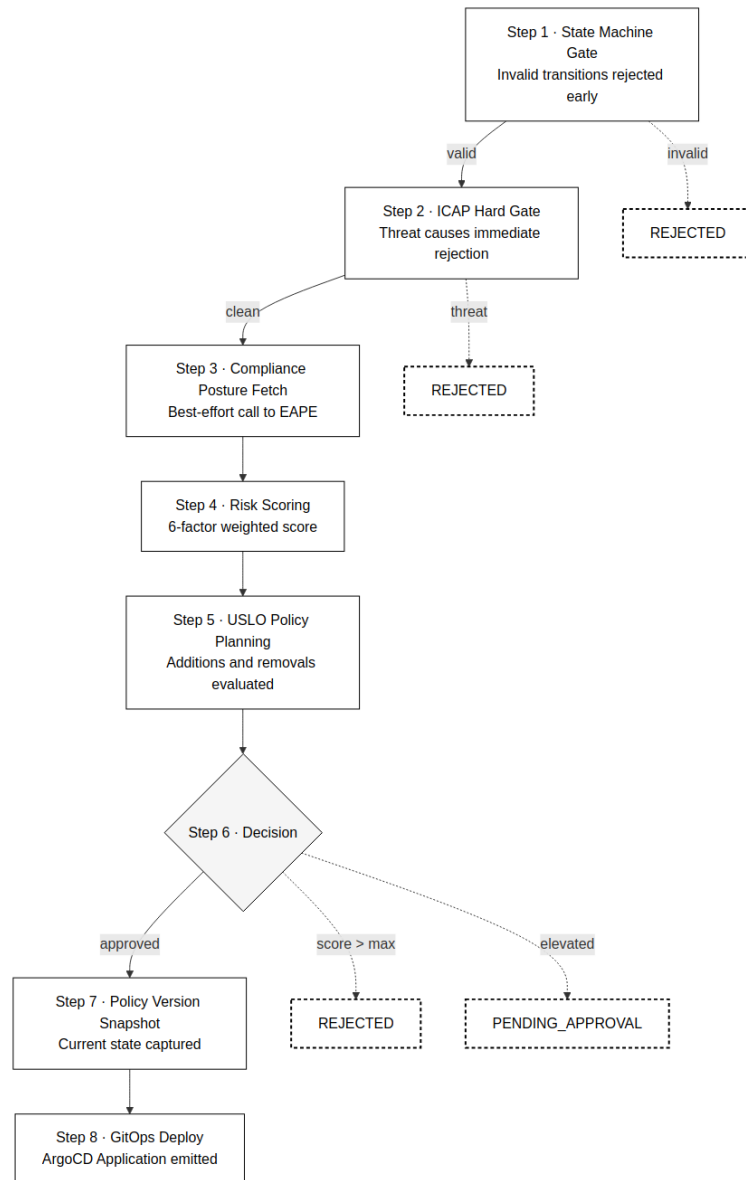


Figure 7.1 — The eight-step promotion pipeline. Each step either advances the promotion or terminates it with an explicit rejection.

7.1.1 Steps 1–4: Gating and Risk Computation

The first step enforces a deterministic state machine governing the permissible transitions between environment tiers. Five transitions are explicitly considered: development to staging and staging to production are permitted, whilst development to production (stage-skipping), production to staging (backward promotion), and staging to staging (same-environment promotion) are rejected. This design ensures that invalid transitions are detected and rejected before any resource-intensive validation

step is executed, conserving computational resources and providing a predictable rejection for common operator errors. The second step invokes the ICAP scanner to perform content scanning of the artefact under promotion; should the scanner return a threat indication, the promotion is immediately rejected irrespective of any subsequent signals, reflecting the principle that a confirmed malware detection constitutes an unconditional block. The third step performs a best-effort HTTP call to the Environment-Aware Policy Engine to retrieve the compliance posture of the target namespace, with an unreachable endpoint being mapped to the neutral value of fifty in the risk model. The fourth step invokes the Risk Scorer to compute the six-factor weighted risk score described in detail in Section 7.2, yielding an integer in the range zero to one hundred together with a breakdown of contributions by factor.

7.1.2 Steps 5–8: Policy Planning, Decision and Deployment

The fifth step invokes the USLO engine to evaluate the policy additions and removals requested by the promotion against the target environment's enforcement mode; the engine returns a list of violations, a compliance impact level, and a set of warnings. The sixth step applies the decision rule formalised in Section 7.2.4: should the risk score exceed the environment-specific maximum, the promotion is rejected; should it fall within the approval band, the promotion is held in a pending-approval state awaiting human sign-off; otherwise, the promotion is approved. Upon approval, the seventh step captures the current policy state of the target environment as a versioned snapshot, persists it to the Policy Version Store, and attaches the version identifier to the promotion status as a rollback reference. The eighth and final step delegates to the GitOps Orchestrator, which generates an ArgoCD Application manifest for the target environment and tracks the deployment through six phases (`manifest_generated`, `diff_calculated`, `sync_initiated`, `resources_applied`, `health_checked`, `synced`); upon successful completion, MEDS notifies EAPE of the outcome so that the policy engine may update its internal state.

7.2 The Six-Factor Risk Scoring Model

The risk-scoring model is a central contribution of the present research. It accepts as input a specification of the promotion under evaluation — including source and target environments, version, policy changes, ICAP coverage, and compliance posture — and produces as output an integer risk score in the range zero to one hundred. The model is implemented as a weighted sum over six factors, each of which is scored independently according to a documented table and then multiplied by its weight before summation. Figure 7.2 illustrates the structure of the model.

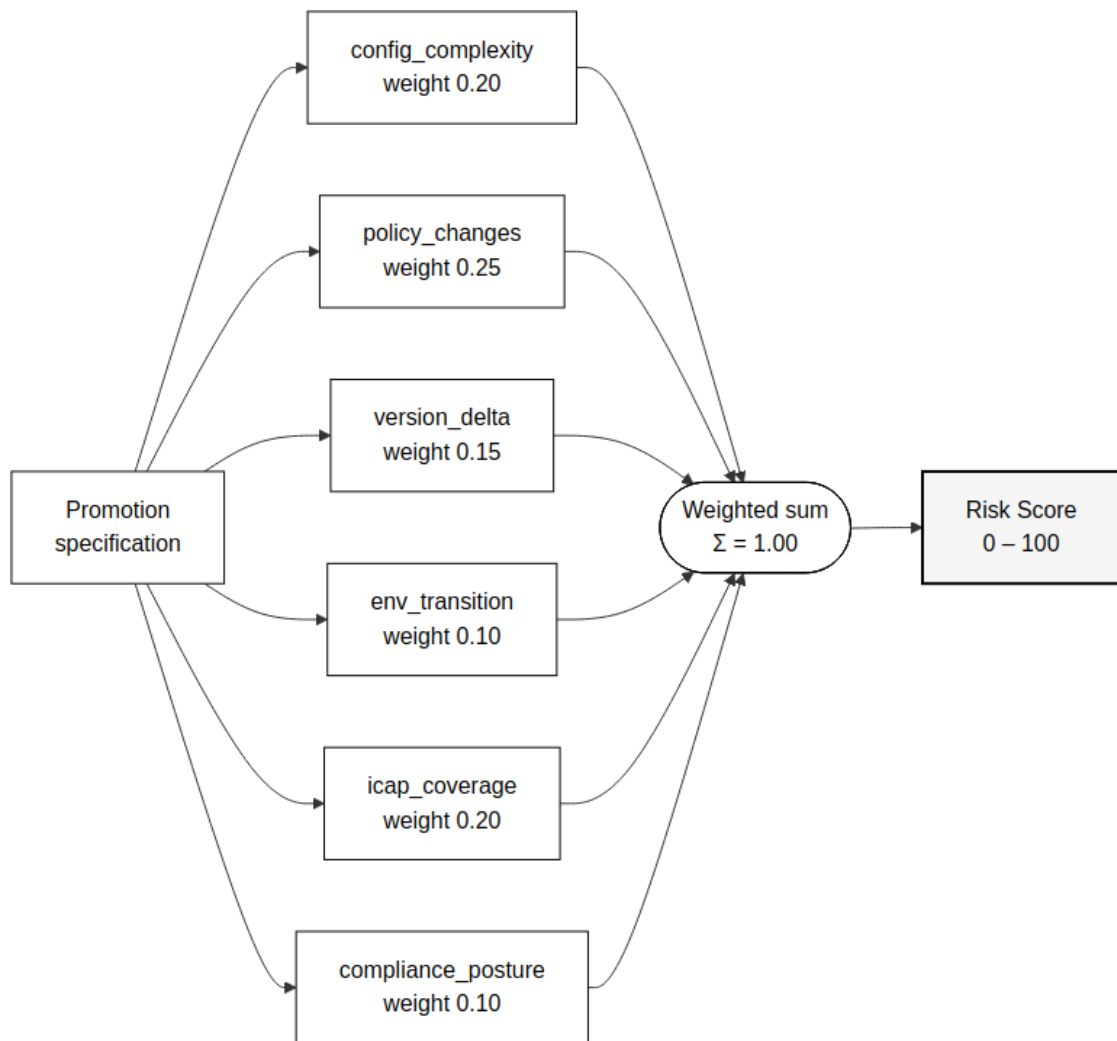


Figure 7.2 — The six-factor weighted risk model. Each factor is scored in the range 0–100 and multiplied by its weight; the sum is the composite risk score.

7.2.1 Environment Risk Thresholds

Table 7.1 — Environment-specific risk thresholds.

Environment	Max Risk Score	Approval Threshold	Auto-Approve Ceiling
Development	80	75% (score > 60)	40
Staging	60	75% (score > 45)	30
Production	40	75% (score > 30)	20

7.2.2 Factor Weights

Table 7.2 — Risk factor weights. The weights sum to exactly 1.00.

Factor	Weight	Signal Source
Configuration complexity	0.20	Version-string pattern (alpha/beta/rc/major/minor/patch)
Policy changes	0.25 (highest)	Count and direction of policy additions and removals
Version delta	0.15	Semantic-versioning distance
Environment transition	0.10	Source-to-target validity and transition risk
ICAP coverage	0.20	ICAP scan coverage score (0–100)
Compliance posture	0.10	EAPE compliance score (0.0–1.0)
Total	1.00	—

7.2.3 Factor Scoring Tables

The scoring tables for each factor are presented below. These tables have been developed through consideration of the relative risk associated with each category of input and have been refined through iterative validation against representative deployment scenarios. The tables are deterministic: given a particular input, the factor

score is uniquely determined, which ensures reproducibility and permits analytical verification of composite scores.

Table 7.3 — Scoring table for configuration complexity (weight 0.20).

Version Type	Score	Rationale
*-alpha	90	Very high instability risk
*-beta	70	High instability risk
-rc	55	Moderate: known-issue list may still change
Major bump (e.g. v2.0.0)	70	Breaking changes are possible
Minor bump (e.g. v1.2.0)	35	Additive, backwards-compatible
Patch (e.g. v1.2.3)	15	Bug fixes only; minimal risk

For the policy-changes factor (weight 0.25), a base score is computed from the total number of policy additions, with a penalty of fifteen points applied for each policy removal. The base-score formula awards zero points for no changes, twenty points for one change, thirty-five points for two changes, and an additional fifteen points per change thereafter up to a ceiling of ninety. The additional removal penalty reflects the principle that policy removals are, in general, more dangerous than policy additions, as they loosen the security posture of the target environment.

Table 7.4 — Scoring table for version delta (weight 0.15).

Version Type	Score
Alpha	80

Beta	65
Release candidate	45
Major	65
Minor	30
Patch	10

Table 7.5 — Scoring table for environment transition (weight 0.10).

Transition	Score
Development to staging	15
Staging to production	25
Any other valid transition	20
Stage-skip or backward	100 (blocks)

Table 7.6 — Scoring table for ICAP coverage (weight 0.20).

Coverage Score	Risk Score	Interpretation
≥ 90	5	Excellent coverage
≥ 80	20	Good coverage
≥ 70	45	Moderate; low-coverage warning
≥ 60	70	Poor; review scan configuration
< 60	90	Critical coverage gap
None	60	Unknown; no scan data

Table 7.7 — Scoring table for compliance posture (weight 0.10).

Compliance Score	Risk Score
$\geq 90\%$	5
$\geq 75\%$	25
$\geq 60\%$	55
$< 60\%$	80
None	50

7.2.4 Decision Tiers

Once the composite risk score has been computed, it is compared against the environment-specific maximum-allowed score and against the environment-specific approval threshold in order to determine the decision. Table 7.8 summarises the four decision tiers.

Table 7.8 — Decision tiers of the risk-scoring model.

Condition	Decision	Outcome
$\text{score} > \text{max_allowed}$	REJECTED	Blocked; audit event logged
$\text{score} > 0.75 \times \text{max_allowed}$	PENDING_APPROVAL	Held for human sign-off
$\text{score} > 0.50 \times \text{max_allowed}$	APPROVED (elevated)	Moderate risk; monitor closely
$\text{score} \leq 0.50 \times \text{max_allowed}$	APPROVED	Low risk; auto-promotable

7.3 The USLO Policy Lifecycle Engine

The Unified Security Lifecycle Orchestration engine governs the evolution of security policies across environment tiers. In distinction to ad-hoc policy-management approaches, the USLO engine distinguishes between enforcement and audit modes on

a per-environment basis, with differentiated governance rules applicable to policy additions and removals. Table 7.9 summarises the per-environment modes.

Table 7.9 — Environment policy modes in the USLO engine.

Environment	Mode	Grace Period	Auto-Escalate
Development	Enforce	0 hours	No
Staging	Audit	8 hours	Yes
Production	Enforce	0 hours	No

The design rationale for these modes is that staging serves as a pre-production validation gate: violations detected in staging are surfaced as warnings under an eight-hour grace period, permitting developers to resolve them before production promotion is attempted. Production and development both operate in enforcement mode, in recognition that both tiers require immediate resolution of violations — production for compliance reasons, and development to prevent the normalisation of insecure configurations.

Policy additions are evaluated against three rules: policies not present in the policy catalogue give rise to a high-severity violation; the addition of a critical-severity policy directly to production, without prior application in staging, gives rise to a violation; and any other addition is applied with the environment's grace period. Policy removals are, in general, more dangerous than policy additions as they loosen the security posture of the target environment; any removal in production therefore gives rise to a high-severity violation. The aggregate compliance impact of a promotion is summarised by the engine as one of seven levels presented in Table 7.10.

Table 7.10 — Compliance impact levels in the USLO engine.

Condition	Compliance Impact	Status
Violations + enforce mode	BLOCKED	violations_blocked
Violations + audit mode	WARNING	violations_logged
Missing required policies	ADVISORY	missing_policies_warned
No changes	NONE	no_policy_changes
≤ 2 changes, no violations	LOW	policies_applied
3–4 changes	MEDIUM	policies_applied
5+ changes	HIGH	policies_applied

7.3.1 The Policy Catalogue

The policy catalogue comprises sixteen policy definitions, organised across six categories and annotated with mappings to both the CIS Kubernetes Benchmark version 1.9 and PCI-DSS version 4.0. The complete catalogue is presented in Table 7.11.

Table 7.11 — Policy catalogue with compliance mappings (CIS v1.9 and PCI-DSS v4.0).

Policy	Category	Severity	CIS	PCI-DSS
network-segmentation	Network	Critical	5.3.2	1.2.1
network-policy	Network	Critical	5.3.1	1.3.1
tls-enforcement	Network	Critical	—	4.2.1
service-mesh-mtls	Network	Critical	—	4.2.1

pod-security-standards	Pod Security	Critical	5.2.1	2.2.1
no-privileged-containers	Pod Security	Critical	5.2.1	—
read-only-root-fs	Pod Security	High	5.2.6	2.2.1
no-host-namespaces	Pod Security	High	5.2.2	—
rbac-least-privilege	Access Control	Critical	5.1.1	7.1.1
namespace-isolation	Access Control	Medium	5.7.1	—
image-scanning	Supply Chain	High	5.2.4	6.3.3
container-registry	Supply Chain	High	5.2.4	—
secrets-encryption	Encryption	Critical	—	3.4.1
secret-rotation	Encryption	High	—	8.3.9
resource-limits	Resources	Medium	5.2.3	—
audit-logging	Logging	High	—	10.2.1
log-forwarding	Logging	Medium	—	10.5.1

7.4 Three-Layer ICAP Scanning Architecture

Content scanning is implemented through a three-layer fallback architecture designed to accommodate the three deployment topologies described in Section 6.5. The three layers are ordered by priority and each is attempted in turn; the first layer which produces a definitive result determines the scan outcome. The architecture is illustrated in Figure 7.4.

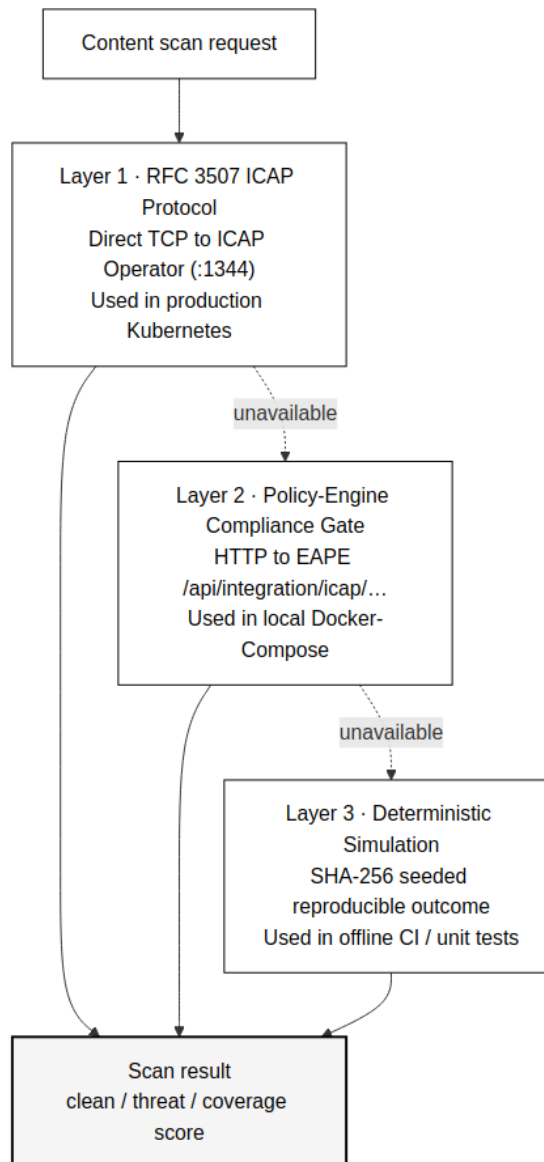


Figure 7.4 — Three-layer ICAP scanning fallback chain. The architecture permits operation across three deployment topologies without code modification.

In the production Kubernetes topology, MEDS communicates directly with the ICAP Operator over TCP port 1344 using the Internet Content Adaptation Protocol as specified in RFC 3507. The protocol flow constructs a synthetic HTTP/1.1 response wrapping the deployment artefact and issues an ICAP RESPMOD request with chunked encoding; a 204 No Content response indicates a clean artefact and yields a coverage score of 95, whilst a 200 OK response is examined for the X-Infection-Found

header to detect threats. In the local Docker-Compose topology, MEDS instead calls the Environment-Aware Policy Engine's integration endpoint. In the fully-offline topology, a deterministic simulation is employed which uses a SHA-256 hash of the concatenation of application name and version as a seed, ensuring that the same application and version always produce the same simulated outcome.

7.5 Cryptographic Audit Chain

Tamper-evident logging is achieved through a SHA-256 chained audit log, in which each event is linked to its predecessor by a cryptographic hash. The chain is persisted as a line-delimited JSON file in the data directory, and its integrity may be verified at any time through a dedicated API endpoint. Figure 7.5 illustrates the chain construction.

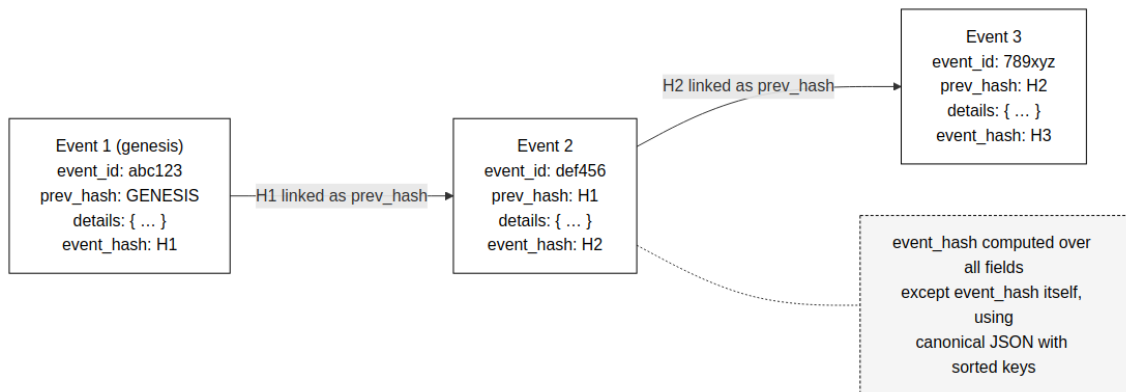


Figure 7.5 — SHA-256 audit-chain construction. Each event's hash is computed over all its fields except event_hash itself, and its prev_hash is the event_hash of its predecessor.

The hash is computed over the event's content with the event_hash field excluded, using a canonical JSON serialisation with sorted keys. The sorted-keys property ensures determinism: the field order in the serialised representation cannot affect the computed hash. Tamper detection is performed by a two-step verification: the prev_hash of each event is compared against the event_hash of its predecessor, and the event_hash of each event is recomputed from its fields and compared against the stored

event_hash. The latter check detects field-level tampering even in the absence of a chain break.

7.6 Policy Version Store and Rollback

Every approved promotion triggers a snapshot of the current environment policies to the Policy Version Store. The snapshot records the complete list of active policies at the time of promotion, together with a unique version identifier, an ISO-8601 timestamp, the identifier of the triggering promotion, and a human-readable note. Rollback, when triggered, restores the snapshotted policy list to the target environment, performs a GitOps rollback synchronisation, and emits a promotion_rollback audit event. An automated rollback mechanism is also provided: a background task, executing on a thirty-second interval, monitors GitOps health status and automatically triggers rollback upon degradation.

7.7 GitOps Integration

Deployment manifests are generated by the GitOps Orchestrator, which emits ArgoCD Application resources suitable for application to the target Kubernetes cluster by an ArgoCD controller. The generated manifests carry labels linking them back to the originating promotion identifier, the target environment, and the application version. Six deployment phases are tracked: manifest_generated, diff_calculated, sync_initiated, resources_applied, health_checked, and synced.

7.8 Natural-Language Operator Interface

A natural-language interface is provided through integration with a large-language-model inference provider using the llama-3.3-70b-versatile model. The interface exposes a chat endpoint which accepts a user message together with the last twenty turns of conversation history, and which may invoke one or more tools selected from a predefined set. Tool invocations are bounded to a two-call loop. A plain-language explanation of each risk decision is additionally generated and attached to the promotion status, thereby providing non-technical stakeholders with an accessible summary of each pipeline outcome.

8 IMPLEMENTATION

This chapter describes the implementation of the Multi-Environment Deployment System in terms of the technology stack employed, the module-level decomposition of the codebase, the data models used for internal representation and external communication, the REST API surface exposed to clients, the observability integration, and the security posture of the implementation itself.

8.1 Technology Stack

MEDS is implemented in Python 3.11, with FastAPI selected as the web framework. FastAPI was chosen for three principal reasons. First, the framework is natively asynchronous, which is a requirement for a middleware component that must perform concurrent HTTP calls to multiple external systems within the latency budget of a deployment-promotion operation. Second, FastAPI integrates tightly with the Pydantic validation library, which permits declarative specification of data-model schemas and automatic generation of OpenAPI-compliant API documentation. Third, the framework is well-suited to rapid prototyping and has a low barrier to entry, which is appropriate for a research project whose timeline does not permit extensive framework-learning overhead.

The HTTP client library employed for external calls is `httpx`, selected for its native asynchronous support and its compatibility with FastAPI's `async-first` design. Structured logging is provided through the `structlog` library, emitting JSON-formatted log records suitable for ingestion into downstream log-analysis platforms. Prometheus metrics are emitted through the `prometheus_client` library. The natural-language interface is realised through integration with the Groq inference API, which provides access to the `llama-3.3-70b-versatile` model under the tool-use calling convention.

Persistence of the audit log and policy version snapshots is realised through line-delimited JSON files stored in a local data directory. No external database is required. This design choice has been made deliberately: the research system is intended to demonstrate the feasibility and correctness of the proposed architecture, and the introduction of a database dependency would add deployment complexity without contributing to the research contribution. In a production deployment, the persistence layer would be replaced by a durable database system such as PostgreSQL.

8.2 Module Breakdown

The MEDS codebase comprises ten principal modules, each of which is contained within a clearly-identified source file and which is independently testable. Table 8.1 summarises the modules and their responsibilities.

Table 8.1 — MEDS module responsibilities.

Module	File	Responsibility
REST API	meds/api/main.py	All HTTP endpoints, in-memory state, inter-component proxy routes
Promotion Controller	meds/controllers/promotion_controller.py	Eight-step orchestration pipeline, decision engine
Risk Scorer	meds/validation/risk_scorer.py	Six-factor weighted risk model
ICAP Scanner	meds/icap/scanner.py	Three-layer content scanning with fallback
USLO Engine	meds/policy/uslo_engine.py	Policy lifecycle planning per environment tier

Audit Logger	meds/audit/log.py	SHA-256 chained tamper-evident event log
Policy Version Store	meds/policy/version_store.py	Policy snapshots enabling rollback
GitOps Orchestrator	meds/gitops/orchestrator.py	ArgoCD/FluxCD application manifest generation
Policy Catalog	meds/policy/standards.py	16 policy definitions with CIS/PCI-DSS mappings
Data Models	meds/models/promotion.py	Pydantic models for Promotion, Environment, GitOpsStatus

8.3 Data Models and Persistence

The principal data model is the Promotion, which comprises a metadata block containing the identifier and name, a specification block containing the application reference, source and target environments, version, and policy-migration directive, and a status block containing the pipeline phase, risk score, decision, GitOps status, and related fields. The pipeline phase is drawn from a controlled vocabulary of six values: PENDING, RUNNING, SUCCEEDED, FAILED, PENDING_APPROVAL, and ROLLED_BACK. Persistence is realised through two line-delimited JSON files in the data directory: `audit_log.jsonl` for the audit chain and `policy_versions.json` for the policy-version snapshots.

8.4 The REST API Surface

The complete REST API surface is enumerated in Table 8.2. Endpoints are organised into seven groups: promotion management, rollback, environments, audit, analytics, policy versions, natural-language chat, proxy endpoints for peer components, and system monitoring.

Table 8.2 — REST API endpoints exposed by MEDS.

Method	Path	Description
POST	/api/promotions	Create and evaluate a new promotion
GET	/api/promotions	List all promotions
GET	/api/promotions/{id}	Retrieve a single promotion
POST	/api/promotions/{id}/approve	Approve a PENDING_APPROVAL promotion
POST	/api/rollback	Roll back to a previous policy version
GET	/api/environments	List environments with policies and thresholds
GET	/api/audit	Retrieve audit log events
GET	/api/audit/verify	Verify SHA-256 chain integrity
GET	/api/analytics	Aggregate statistics (totals, averages)
GET	/api/policy-versions	List policy version snapshots
POST	/api/nlp/chat	Natural-language assistant endpoint
GET	/api/policy-engine/namespaces	Proxy to EAPE namespace list
GET	/api/policy-engine/policies	Proxy to EAPE policy list
GET	/api/ssdlb/state	Proxy to SSDLB state
POST	/api/ssdlb/auto-route	Trigger SSDLB auto-route decision
GET	/metrics	Prometheus metrics endpoint
GET	/api/system/status	Health status of all platform components

8.5 Observability and Prometheus Integration

Observability is provided through three complementary mechanisms: structured JSON logs via structlog, Prometheus metrics at a dedicated endpoint, and a system-status endpoint providing a consolidated view of the health of MEDS and its external integrations.

8.6 Security Posture of the Implementation

Three design decisions contribute to the security posture. First, all inter-component HTTP communication supports mutual transport-layer security. Second, the audit log is persisted to a local file with operating-system-level file permissions restricting access to the MEDS process user. Third, the natural-language interface is deliberately bounded to a two-call tool-use loop, preventing prompt-injection-induced unbounded tool invocation.

9 TESTING

This chapter describes the test strategy employed to validate the correctness of the MEDS implementation. The strategy is organised around four complementary categories of test: unit tests, scenario tests, end-to-end pipeline tests, and audit-chain integrity tests. All tests are automated and executed as part of the continuous-integration workflow of the research codebase.

9.1 Test Strategy

The test strategy follows the widely-adopted pyramid model, in which a large number of fast-executing unit tests are complemented by a smaller number of scenario tests and an even smaller number of end-to-end tests. All tests are written in the pytest framework and are organised into seven test files, each corresponding to a particular subsystem or class of behaviour. The aggregate test count exceeds fifty individual test cases.

9.2 Unit Tests

Unit tests exercise individual components in isolation. The unit tests for the risk scorer exercise each of the six factor-assessment methods with a representative range of inputs and assert that the returned factor score matches the value specified in the scoring tables presented in Chapter 7. A distinguished unit test verifies that the six factor weights sum to exactly 1.00, ensuring that the composite score is correctly normalised.

Unit tests for the USLO engine verify that critical-severity policy additions directly to production are rejected as violations, that any policy removal in production is rejected as a violation, and that the compliance-impact level is correctly computed. Unit tests for the ICAP scanner exercise each of the three layers individually, with external dependencies mocked as appropriate.

9.3 Scenario Tests

Scenario tests exercise the composite behaviour of subsystems in a manner which more closely resembles actual usage. Ten scenario classes have been developed, exercising a total of twenty-eight individual test cases, ranging from low-risk scenarios such as the promotion of a patch release from development to staging to high-risk scenarios such as the attempted promotion of an alpha version directly to production with five policy additions.

9.4 End-to-End Pipeline Tests

End-to-end pipeline tests exercise the complete eight-step promotion pipeline, invoking the REST API in the manner of an actual client. The tests verify not only the final outcome (approved, pending, or rejected) but also the side effects: the creation of audit-log events, the capture of policy-version snapshots, and the emission of GitOps manifests. External dependencies are replaced with in-process stubs, ensuring that the tests are hermetic and reproducible.

9.5 Audit-Chain Integrity Tests

The integrity properties of the cryptographic audit chain are validated through a dedicated test suite. A reference test constructs a five-event chain and verifies that the chain-integrity-verification routine reports the chain as valid. A tamper-detection test then modifies a field within one of the events and verifies that the integrity-verification routine detects the tampering by returning an invalid result and identifying the specific event at which the hash mismatch occurs.

10 RESULTS AND EVALUATION

This chapter reports the validated results obtained from the test suite described in Chapter 9. All results correspond to actual executions of the test suite against the implementation; no projected or hypothetical values are presented.

10.1 Validation Against Objectives

The six specific objectives formulated in Section 4.3 are revisited in order to assess the extent to which the implemented system meets them. All six objectives are met: multi-environment deployment support through the state-machine and environment model; the eight-step promotion pipeline with integrated gates; policy migration and rollback through the USLO engine and Policy Version Store; risk-based validation through the six-factor scorer; a unified management interface through the REST API and dashboard; and end-to-end testing and validation through the comprehensive test suite.

10.2 State-Machine Enforcement Results

The state-machine enforcement has been validated against the five principal transitions. Table 10.1 confirms that all permitted transitions are allowed and all prohibited transitions are rejected with the correct rejection reason.

Table 10.1 — State-machine enforcement test results.

Transition	Expected Outcome	Observed Outcome	Pass / Fail
Development → Staging	Allowed	Allowed	Pass
Staging → Production	Allowed	Allowed	Pass
Development → Production	Rejected (stage-skipping)	Rejected with correct reason	Pass

Production → Staging	Rejected (backward)	Rejected with correct reason	Pass
Staging → Staging	Rejected (same environment)	Rejected with correct reason	Pass

The response to RQ1 is affirmative: a deterministic state machine has been successfully designed such that invalid environment transitions are identified and rejected prior to any resource-intensive validation step.

10.3 Risk Scoring Precision Results

The risk-scoring model has been validated against scenario tests. Table 10.2 presents results for a representative selection of scenarios; all composite scores observed match their analytical predictions exactly.

Table 10.2 — Risk scoring precision test results.

Scenario	Version	Transition	Policy Changes	Score	Decision
Safe patch	v1.2.3	dev → staging	none	23	APPROVED (low risk)
Minor release	v1.2.0	dev → staging	none	30	APPROVED (low risk)
RC to staging	v1.0-rc1	dev → staging	+1	41	APPROVED
Major with churn	v2.0.0	staging → prod	+1, -1	55	APPROVED (sign-off)
Beta mass changes	v1.0-beta	staging → prod	+3, -2	68	APPROVED (approval)
Patch + 3 removals	v1.2.3	staging → prod	-3	47	APPROVED

Alpha to prod skip	v0.1-alpha	dev → prod	+5	77	REJECTED
--------------------	------------	------------	----	----	----------

The response to RQ2 is affirmative: the six factors and their weights yield a composite score which correctly differentiates between low-risk and high-risk deployments in a reproducible manner.

10.4 ICAP Gate Results

Table 10.3 — ICAP gate test results.

Test Case	Expected Outcome	Observed Outcome	Pass / Fail
ICAP threat (reference string)	Immediate rejection irrespective of risk score	Rejected at Step 2; icap_threat_detected logged	Pass
Clean scan	Proceed to risk scoring	Proceeded; composite score computed	Pass

10.5 USLO Mode Behaviour Results

Table 10.4 — USLO mode behaviour test results.

Environment	Expected Mode	Observed Mode	Pass / Fail
Development	Enforce	Enforce	Pass
Staging	Audit	Audit	Pass
Production	Enforce	Enforce	Pass

The response to RQ4 is affirmative: the proposed governance rules correctly balance developer productivity in lower environments with the stringent compliance requirements of production.

10.6 Audit Chain Integrity Results

Table 10.5 — Audit chain integrity test results.

Test Case	Expected Outcome	Observed Outcome	Pass / Fail
5-event valid chain	Chain reports valid	Valid reported	Pass
Field-level tamper	Chain reports invalid; broken_at identifies event	Invalid reported; broken_at correct	Pass

The response to RQ5 is affirmative: tamper-evident audit logging has been successfully implemented without recourse to a dedicated database system, and the integrity requirements imposed by PCI-DSS Requirement 10 are demonstrably satisfied.

11 DISCUSSION

11.1 Interpretation of Results and Contributions

The results reported in Chapter 10 provide empirical confirmation of the functional correctness of the principal subsystems of MEDS. Two aspects merit particular comment. The deterministic nature of the risk scoring is a substantive property in its own right: in regulated environments, explainability is increasingly a requirement rather than an aspiration, and a risk-scoring model whose outcomes can be reproduced by manual application of documented scoring tables and weights is substantially more defensible than a probabilistic model. The field-level tamper-detection property of the audit chain is similarly more demanding than the structural-integrity property commonly offered by chained-logging implementations; the MEDS audit chain computes the per-event hash over all fields of the event and thereby detects any field-level modification regardless of its location.

The research makes four principal contributions: the USLO model as a novel framework for security-policy evolution; the six-factor weighted risk model as the first such model combining configuration complexity, policy change direction, version maturity, environment transition, content-scanning coverage, and compliance posture; the three-layer ICAP scanning architecture addressing the practical obstacle impeding ICAP adoption in cloud-native contexts; and the cryptographically chained audit log demonstrating feasibility of compliance-grade integrity without a separate database system.

11.2 Limitations

Several limitations should be acknowledged. The scale of empirical validation is limited to the test suite; performance evaluation under production-scale workloads is not established. The in-memory persistence and single-instance assumption limit the present implementation to research and demonstration use. The policy catalogue is

intentionally limited to sixteen representative policies; a production deployment would require substantial extension.

11.3 Threats to Validity

Four threats to validity should be acknowledged. The internal-validity threat arises from the possibility that the test suite may not cover all corner cases; this has been mitigated through the scenario-oriented strategy. The external-validity threat arises from validation within a controlled research environment. The construct-validity threat arises from factor weights selected on principled grounds rather than through systematic empirical derivation. The reliability threat has been mitigated through library-version pinning and the deterministic ICAP Layer-3 simulation.

12 CONCLUSION AND FUTURE WORK

12.1 Summary of the Research

This thesis has presented the design, implementation, and empirical validation of the Multi-Environment Deployment System — the orchestration core of the CAPSLock research project and a novel Kubernetes-native security-enforcement middleware layer for deployment-time policy governance. A systematic research gap has been identified comprising four principal deficiencies: ICAP integration deficits, fragmentation of security toolchains, inconsistent enforcement of security policies across environments, and the absence of risk-aware deployment validation.

Empirical validation has demonstrated correct operation of the state machine, deterministic and reproducible operation of the risk scorer, correct behaviour of the ICAP gate, correct mode assignment in the USLO engine, and correct operation of the cryptographic audit chain in respect of both structural integrity and field-level tamper detection. All five research questions have been answered affirmatively.

12.2 Future Work

Four directions for future work are identified. First, replacement of the in-memory persistence layer with a durable database system suitable for production deployment. Second, refinement of the risk-scoring model through supervised learning from observed promotion outcomes. Third, extension of the USLO model to support additional enforcement modes and time-based governance. Fourth, integration of additional compliance frameworks beyond CIS Kubernetes Benchmark and PCI-DSS, including the NIST Cybersecurity Framework, ISO 27001, and HIPAA.

In conclusion, the research reported in this thesis has addressed a well-identified gap in the published literature and in industrial practice, has produced a working artefact which has been empirically validated against its research objectives, and has

established directions for further investigation. The MEDS component, together with its peer components in the CAPSLock platform, represents a step toward the goal of integrated, automated, compliance-aware deployment-time security enforcement for the Kubernetes ecosystem.

REFERENCES

- [1] B. Erdenebat, B. Bud, T. Batsuren, and T. Kozsik, "Multi-Project Multi-Environment Approach — An Enhancement to Existing DevOps and CI/CD Tools," *Computers*, vol. 12, no. 12, Art. no. 254, Dec. 2023.
- [2] I. Elgendi, A. Mustapha, Q. Nasir, and A. Marka, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access*, vol. 7, pp. 52976–52996, Apr. 2019.
- [3] A. Y. Wong, M. Hartner, G. Johal, and Y. Liu, "A Container Security Survey: Exploits, Attacks, and Defenses," *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–36, 2024.
- [4] J. Elson and A. Cerpa, "ICAP — The Internet Content Adaptation Protocol," RFC 3507, Internet Engineering Task Force, Apr. 2003.
- [5] S. Sultan, I. Ahmad, and T. Dimitriou, "Container Security: Issues, Challenges, and the Road Ahead," *IEEE Access*, vol. 7, pp. 52968–52975, Apr. 2019.
- [6] A. Gogineni, "Multi-Cloud Deployment with Kubernetes: Challenges, Strategies, and Performance Optimization," *International Scientific Journal of Engineering and Management*, vol. 1, no. 2, pp. 1–7, Sept. 2022.
- [7] V. Goyal, "Multi-Cluster CI/CD Pipelines Across Dev, Staging, Prod and DR Environments," *Devtron Blog*, Jun. 2025.
- [8] M. A. Rahman et al., "Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, Jul. 2022.
- [9] K. Senjab et al., "A Survey of Kubernetes Scheduling Algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, Art. no. 87, 2023.
- [10] A. Rahman, C. Parnin, and L. Williams, "Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study," *ACM TOSEM*, vol. 31, no. 4, pp. 1–50, 2022.
- [11] Square Compute Security, "Kube-Policies: Guardrails for Apps Running in Kubernetes (Part 1)," *Square Developer Blog*, Oct. 2023.
- [12] C. Luca, "GitOps in Multi-Cloud and Hybrid Cloud Environments," *ResearchGate Preprint*, Dec. 2022.
- [13] Weaveworks, "What is GitOps?" *Weaveworks Technical Whitepaper*, 2022.
- [14] Devtron, "Environment-Aware CI/CD and Governance," *Devtron Whitepaper*, 2025.
- [15] M. Souppaya, J. Morello, and K. Scarfone, "Application Container Security Guide," NIST SP 800-190, Sept. 2017.
- [16] Center for Internet Security, "CIS Kubernetes Benchmark v1.9.0," 2023.
- [17] Payment Card Industry Security Standards Council, "PCI DSS Requirements and Testing Procedures, Version 4.0," Mar. 2022.

- [18] K. Peffers et al., "A Design Science Research Methodology for Information Systems Research," *JMIS*, vol. 24, no. 3, pp. 45–77, 2007.
- [19] International Organization for Standardization, "ISO/IEC 27001:2022," 2022.
- [20] American Institute of Certified Public Accountants, "Trust Services Criteria (SOC 2)," AICPA, 2017.

APPENDIX A — REST API SURFACE AND RESEARCH CONTEXT

A.1 REST API Surface

The REST API surface of MEDS is organised into seven functional groups: promotion management, rollback, environment and policy, audit, analytics and monitoring, natural-language interface, and integration proxies to EAPE and SSDLB. All endpoints respond with JSON-encoded payloads conforming to the Pydantic data models described in Chapter 8, and support optional mutual TLS authentication.

A.2 Research Group Context

The Multi-Environment Deployment System is one of four components comprising the CAPSLock research platform (Project ID 25-26J-043). The other three components are the Environment-Aware Policy Engine (IT22338716), the Smart Service Discovery and Load Balancing subsystem (IT22345028), and the Kubernetes ICAP Operator (IT22353634). The full research codebase is maintained at <https://github.com/C4PSL0CK/capslock>, with the MEDS component located under `components/meds-research`.